SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1010	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S.BHAVYA	**Reg. No.:**	192524376
Date:	28.10.26	**Duration:**	60 minutes

Code of Conduct

I Bhavya(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Bhavya S*

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

PERSONAL EXPENSE AND BUDGET MANAGEMENT SYSTEM



1. Analyze System Requirements

1.1 Introduction

The Personal Expense and Budget Management System is a software application designed to help users manage their personal finances efficiently. It allows users to record income and expenses, categorize spending, create budgets, receive alerts, and view financial reports.

Managing finances manually can lead to errors, forgotten transactions, and overspending. This system provides a centralized platform to maintain financial records and monitor spending patterns. It helps users understand their financial position, control unnecessary expenses, and improve their savings.

The system can be developed as a web application accessible through a browser. It stores financial information in a database and processes the data to generate budget summaries, alerts, and reports.

1.2 System Objectives

The main objectives of the system are:

1. To record and manage income and expenses digitally.
2. To categorize expenses for better financial analysis.
3. To create monthly budgets and monitor spending limits.
4. To notify users when their spending approaches or exceeds their budgets.
5. To generate financial reports showing income, expenses, and savings.
6. To protect financial information and maintain accurate records.
7. To support future enhancements without redesigning the entire system.

1.3 Functional Requirements

Functional requirements describe the operations performed by the system.

1. User Registration and Login: Users can register, log in, and securely access their personal financial information.

2. Income Management: Users can add, edit, and delete income records, including salary, allowances, and other earnings.

3. Expense Management: Users can record expenses with the amount, date, category, and description.

4. Expense Categorization: Expenses are grouped into categories such as food, transport, education, shopping, and entertainment.

5. Budget Management: Users can set monthly or category-wise spending limits and view their remaining budget.

6. Alert Management: The system generates notifications when spending reaches a specified threshold or exceeds the budget.

7. Financial Reports: Users can view daily, weekly, and monthly summaries of income, expenses, savings, and category-wise spending.

1.4 Non-Functional Requirements

Non-functional requirements define the quality and performance expected from the system.

- Scalability: The system should support an increasing number of users and financial records.
- Security: User credentials and financial data must be protected against unauthorized access.
- Performance: Transactions and financial reports should be processed within a reasonable time.
- Reliability: The system must maintain accurate records and prevent inconsistent updates.
- Maintainability: Individual modules should be modified without affecting the entire application.
- Availability: The system should minimize downtime and provide recovery mechanisms.
- Usability: The interface should be simple and easy to understand.
- Data Integrity: The system must validate inputs and maintain consistency between transactions and budgets.



Fig; Non-Functional Requirements

1.5 Key Quality Attributes

Quality attribute	Requirement
Scalability	Support increasing users and transactions.
Security	Protect private financial information.
Performance	Generate reports efficiently.
Reliability	Maintain accurate financial records.
Maintainability	Allow easy modification of modules.
Availability	Minimize service interruptions.

These requirements influence the selection of the software architecture. A layered architecture separates user interaction, business processing, and data storage, helping the system satisfy its functional and non-functional requirements.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Three-Tier Layered Architecture

The proposed system uses a Three-Tier Layered Architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

Each layer performs a specific responsibility and communicates with other layers through defined interfaces. This separation makes the system easier to develop, test, maintain, and extend.

2.2 Presentation Layer

The Presentation Layer provides the interface through which users interact with the application. It can be developed using HTML, CSS, and JavaScript.

Its responsibilities include:

- Displaying the login page and dashboard.
- Providing forms to enter income and expenses.
- Allowing users to create and manage budgets.
- Displaying budget alerts and notifications.
- Showing financial reports through tables and charts.

The presentation layer sends requests to the application layer using REST APIs. It displays the results returned by the backend.

2.3 Application Layer

The Application Layer contains the business logic of the system. It processes user requests, validates information, performs financial calculations, and communicates with the database.

It consists of the following modules:

- Authentication Module: Verifies user credentials and controls access.
- Income and Expense Module: Records, updates, and categorizes transactions.
- Budget Management Module: Calculates spending and remaining balances.
- Alert Module: Checks budget thresholds and generates notifications.
- Report Module: Generates financial summaries and spending reports.

This layer ensures that financial rules are applied consistently before information is stored or displayed.

2.4 Data Layer

The Data Layer stores and retrieves financial information. A relational database such as MySQL can be used.

The database contains the following tables:

1. Users – stores user account details.
2. Income – stores income records.
3. Expenses – stores expense details.
4. Categories – stores expense categories.
5. Budgets – stores budget amounts and limits.
6. Notifications – stores alert information and delivery status.

The database maintains relationships between users, transactions, and budgets. Database constraints and transactions help preserve data consistency.

2.5 Justification for Selecting Layered Architecture

The Three-Tier Layered Architecture is suitable because the system has clearly defined responsibilities involving user interaction, financial calculations, and data storage.

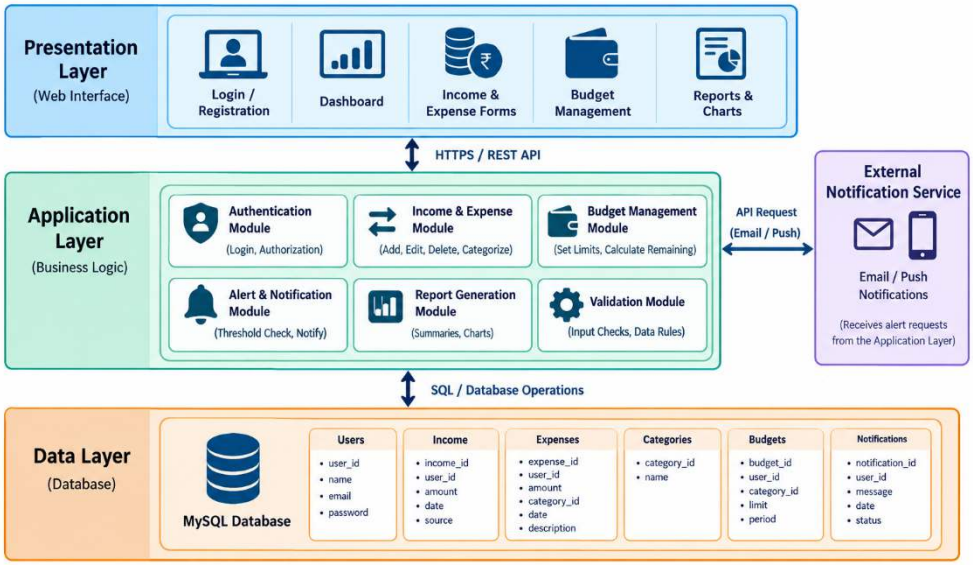
Its major advantages are:

- Separation of concerns: Each layer has a specific responsibility.
- Maintainability: Changes can be made to individual modules.
- Security: Database access can be restricted to the application layer.
- Testability: Each layer can be tested independently.
- Reusability: Backend services can support web and mobile interfaces.
- Extensibility: New features can be added without redesigning the complete system.

A microservices architecture could provide independent scaling but would introduce additional deployment and communication complexity. Therefore, a layered architecture provides a practical balance between simplicity, security, and future growth for this project.

3. Draw the Software Architecture Diagram

3.1 Architecture Diagram



3.2 Explanation of the Diagram

The architecture consists of three main layers and an external notification service.

The Presentation Layer accepts user input and displays financial information. It communicates with the Application Layer through HTTPS-based REST APIs.

The Application Layer processes requests through the authentication, income and expense, budget management, alert, and report modules. It applies validation rules and performs financial calculations.

The Data Layer stores user details, transactions, categories, budgets, and notification records. The application layer accesses this information through database operations.

The external notification service delivers email or push notifications when the application identifies a budget-related event.

The arrows represent communication between components. The layered arrangement demonstrates the separation of responsibilities and prevents the user interface from directly accessing the database.

3.3 Technology Selection

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Java with Spring Boot
API communication	REST API and HTTPS
Database	MySQL
Data access	JDBC or Spring Data JPA
Reports	Chart.js
Testing	JUnit and API testing tools

These technologies are proposed implementation choices. Other equivalent technologies can be used without changing the overall architecture.

4. Explain Components and Their Interactions

4.1 User Interface Component

The User Interface provides access to all application features. It includes login pages, transaction forms, budget settings, dashboards, and reports.

It collects user input and sends requests to the backend. The results returned by the application are displayed as tables, charts, and messages.

4.2 Authentication Component

The Authentication Component verifies user credentials and controls access to financial information. It ensures that users can access only their own records.

Passwords should be stored using secure password hashing rather than plain text. Authentication and authorization checks must be applied to all protected operations.

4.3 Income and Expense Component

This component manages financial transactions. It validates the entered amount, date, and category before storing the transaction in the database.

Users can add, edit, delete, and retrieve records. The component also supplies transaction data to the budget and reporting modules.

4.4 Budget Management Component

The Budget Management Component allows users to set monthly and category-wise spending limits. It calculates the amount spent and the remaining balance.

For example:

- Monthly budget: ₹20,000
- Total expenses: ₹14,000
- Remaining budget: ₹6,000

Remaining Budget = Total Budget − Total Expenses

The calculated values are displayed on the dashboard and used by the alert module.

4.5 Alert and Notification Component

The Alert Component compares current spending with the configured budget threshold. If the spending reaches or exceeds the threshold, it generates an alert.

For example, if the monthly budget is ₹10,000 and the alert threshold is 80%, the system can notify the user when spending reaches ₹8,000.

The component may display an in-app notification or use an external email or push notification service. It should also prevent unnecessary repeated alerts for the same event.

4.6 Report Generation Component

The Report Generation Component retrieves financial records and calculates summaries.

It generates daily, weekly, and monthly reports, category-wise expense comparisons, budget utilization summaries, and savings information.

Savings = Total Income − Total Expenses

The results are returned to the Presentation Layer and displayed using tables, charts, or dashboard summaries.

4.7 Database Component

The database stores and retrieves user accounts, transactions, budgets, categories, and notifications. It maintains persistent records even after users log out.

Database constraints, transactions, and indexes help improve consistency and query performance. Regular backups and tested recovery procedures protect against data loss.

4.8 Component Interaction and Workflow

The system follows this workflow:

1. The user logs in through the web interface.
2. The Authentication Module verifies the credentials.
3. The user records income or expenses.
4. The Application Layer validates the information and stores it in the database.
5. The Budget Module recalculates spending and the remaining budget.
6. The Alert Module checks whether a configured threshold has been reached.
7. If required, the system generates a notification.
8. The Report Module retrieves financial records and calculates summaries.
9. The interface displays the updated budget, alerts, and reports.

This workflow ensures that user requests are validated and processed through the application layer before the results are returned.

5. Discuss Quality Attributes

5.1 Scalability

Scalability is the ability of the application to handle increasing numbers of users and transactions.

The layered architecture supports scalability by separating the user interface, business logic, and data storage. If demand increases, additional application instances can be deployed behind a load balancer.

Database indexing and optimized queries help retrieve financial records efficiently. Background processing can also be introduced for large reports or high volumes of notifications.

These improvements allow the system to expand without replacing the entire architecture.

5.2 Security

Security is important because the application stores private financial information.

The system uses authentication, authorization, HTTPS communication, input validation, and controlled database access. Passwords should be stored as secure hashes, and database credentials should not be exposed in source code.

The backend must verify that each user is authorized to access the requested records. Sensitive information should not be unnecessarily stored in application logs.

These measures help protect against unauthorized access, data exposure, and accidental modification.

5.3 Performance

Performance refers to how quickly the system processes requests and displays information.

The application can improve performance through database indexing, optimized queries, pagination, and efficient report calculations.

For example, indexes on user IDs and transaction dates can help retrieve relevant records more quickly. Report queries can calculate totals directly from the database instead of loading every transaction into the application.

Performance testing should be conducted with different numbers of users and financial records to identify processing delays.

5.4 Reliability

Reliability is the ability of the system to operate correctly and preserve accurate information.

The application validates transaction details before storage. Database transactions help ensure that related changes are completed successfully or rolled back when an error occurs.

Regular backups and tested restoration procedures support data recovery. Error handling and logging help developers identify problems and correct them.

These measures reduce the risk of inconsistent budgets, missing transactions, and incorrect reports.

5.5 Maintainability

Maintainability is the ease with which the application can be updated and corrected.

The layered architecture separates user-interface code, business logic, and database operations. Therefore, changes to the report module can be made without redesigning the login interface.

Reusable services, clear naming, modular code, documentation, and automated tests further improve maintainability.

The architecture also allows developers to work on separate modules with fewer conflicts.

5.6 Availability

Availability is the ability of users to access the system when required.

The application can improve availability through reliable hosting, health monitoring, database backups, and recovery procedures. Multiple application instances can reduce dependence on a single application server.

Failed notification requests can be retried using controlled mechanisms. The application should also display appropriate messages when a service is temporarily unavailable.

Availability depends on the deployment environment and external services, so regular monitoring and recovery testing are necessary.

6. Justify Architectural Decisions

6.1 Rationale Behind the Architecture

The Three-Tier Layered Architecture is selected because it supports secure financial management, accurate budget calculations, efficient reporting, and future maintenance.

The Presentation Layer manages user interaction, the Application Layer applies business rules, and the Data Layer maintains financial records. This separation prevents critical financial calculations from being placed directly in the user interface.

A relational database is suitable because financial records have structured fields and relationships between users, transactions, categories, and budgets.

REST APIs provide a clear communication interface and allow a future mobile application to reuse the existing backend services.

6.2 Comparison with Other Architectures

1. Layered Architecture: Separates the system into logical layers. It is easy to develop, test, and maintain and is suitable for this project.
2. Microservices Architecture: Divides the application into independently deployable services. It supports independent scaling but increases deployment, communication, and monitoring complexity.
3. Event-Driven Architecture: Allows components to react to events such as an expense being recorded or a budget threshold being reached. It is useful for asynchronous notifications but may require additional infrastructure.
4. Selected Approach: The Three-Tier Layered Architecture is the most practical choice for the current project scope because it provides clear separation of responsibilities without unnecessary distributed-system complexity.

6.3 Trade-Offs Considered

The selected architecture offers several advantages, but some limitations must also be considered.

Advantages:

1. Clear separation of responsibilities.
2. Easier testing and maintenance.
3. Controlled database access.
4. Improved code organization.
5. Support for future features.

Limitations:

1. Communication between layers introduces processing overhead.
2. A database failure can affect multiple functions.
3. A single application instance may become a bottleneck as demand grows.
4. External notification services may fail or respond slowly.

Solutions: The system can use optimized database queries, backups, monitoring, error handling, and additional application instances when required. Notification retries should be designed to avoid duplicate alerts.

6.4 Future Enhancements

The proposed architecture can support the following improvements:

- ❖ AI-Based Expense Categorization: Machine learning can suggest expense categories based on transaction descriptions.
- ❖ Spending Prediction: Historical financial records can be analyzed to estimate future spending and identify unusual expenses.
- ❖ Bank API Integration: Authorized bank integrations can import transactions and reduce manual data entry. This requires user consent and secure API access.
- ❖ Mobile Application: A mobile application can use the existing REST APIs to provide access from smartphones.
- ❖ Multi-Currency Support: The system can be extended to manage expenses in different currencies.
- ❖ Advanced Financial Reports: Additional charts can compare monthly spending, savings, and budget utilization.
- ❖ Improved Notifications: Background processing can be introduced if the number of notifications increases.

These enhancements can be added progressively without redesigning the entire system.

6.5 Long-Term Maintainability and Integration

Long-term maintainability depends on modular code, clear interfaces, documentation, testing, and controlled updates.The application should use consistent API conventions and centralized validation rules. Automated tests should verify transaction processing, budget calculations, access permissions, and report accuracy.

Version control and a documented deployment process help reduce errors during updates. External services should be accessed through defined interfaces so that providers can be changed without rewriting the complete application.These practices support future integration with mobile applications, notification services, and authorized financial platforms.

CONCLUSION

The Personal Expense and Budget Management System provides an effective solution for recording income, tracking expenses, creating budgets, receiving alerts, and generating financial reports.The Three-Tier Layered Architecture separates the Presentation Layer, Application Layer, and Data Layer. This design supports security, scalability, performance, reliability, maintainability, and availability.Therefore, the Three-Tier Layered Architecture is suitable for developing a secure, reliable, and maintainable Personal Expense and Budget Management System.